Master 1 — IBAM Année universitaire 2025–2026

Développement à base de composants et services web

API REST de produits

Bases de données, ETL Apache Hop,
API Spring Boot et supervision Prometheus

Rapport de projet — Exercice 1

Groupe	1
Membres	BOUGOUMA Leïla — partie 1 : bases de données OUARMA Landry — partie 2 : ETL WAONGO Osiris — partie 3 : API ZAPRE Malika — partie 4 : supervision KOURA Stéphane — partie 5 : tests
Enseignant	Mr Oumar KY
Date	29 octobre 2026

Code source, scripts et configuration : dépôt Git du projet (branche `main`).

Résumé

Ce rapport présente la réalisation d'une chaîne complète de traitement de données de produits. Des données brutes et volontairement imparfaites, stockées dans MySQL, sont contrôlées, normalisées et dédoublonnées par un ETL Apache Hop, puis chargées dans PostgreSQL. Une API REST Spring Boot expose ces données (consultation paginée, création, modification, suppression) avec un contrat OpenAPI, une validation stricte et un format d'erreur unique. Prometheus supervise l'API en continu. L'ensemble est décrit par un seul fichier Docker Compose et se lance sur n'importe quelle machine disposant de Docker. Sur le jeu de test de 40 lignes, l'ETL charge 29 produits et en rejette 10 avec un motif précis ; l'API est couverte par 27 tests automatisés et par une collection Bruno de 18 requêtes.

Table des matières

1 Introduction **3**
- 1.1 Contexte et objectif . . . 3
- 1.2 Principes directeurs . . . 3

2 Architecture générale **3**
- 2.1 Vue d'ensemble . . . 3
- 2.2 Les services Docker . . . 4
- 2.3 Choix structurants . . . 4

3 Partie 1 — Bases de données et jeu de test **5**
- 3.1 Base source MySQL . . . 5
- 3.2 Base cible PostgreSQL . . . 5
- 3.3 Jeu de test . . . 6

4 Partie 2 — ETL avec Apache Hop **6**
- 4.1 Choix de l'outil . . . 6
- 4.2 Le pipeline de traitement . . . 6
- 4.3 Le workflow d'orchestration . . . 7
- 4.4 Exécution et résultats . . . 7

5 Partie 3 — API REST Spring Boot **8**
- 5.1 Technologies . . . 8
- 5.2 Architecture en couches . . . 8
- 5.3 Les endpoints . . . 9
- 5.4 Validation et gestion des erreurs . . . 9
- 5.5 Contrat OpenAPI . . . 10
- 5.6 Deux améliorations apportées aux réponses . . . 12
- 5.7 Conteneurisation . . . 12

6 Partie 4 — Supervision avec Prometheus **12**
- 6.1 Principe . . . 12
- 6.2 Métriques disponibles . . . 13
- 6.3 Requêtes PromQL utiles . . . 13

7 Partie 5 — Tests et validation **14**
- 7.1 Tests unitaires du service . . . 14
- 7.2 Tests d'intégration de l'API . . . 14
- 7.3 Collection Bruno . . . 15
- 7.4 Collection Postman . . . 15
- 7.5 Test de bout en bout . . . 15
- 7.6 Script automatisé de bout en bout . . . 17

8 Difficultés rencontrées et solutions **18**

9 Organisation du travail **18**

10 Limites et perspectives **19**

11 Conclusion **19**

A Lancer le projet **20**

B Structure du dépôt **20**

C Exemples de requêtes et de réponses **20**

1 Introduction

1.1 Contexte et objectif

Dans une organisation, les données de produits proviennent souvent de sources hétérogènes et peu fiables : champs vides, prix négatifs, catégories écrites de dix façons différentes, doublons. Les exposer telles quelles à des applications clientes les rend inutilisables. Ce projet, réalisé dans le cadre du module ń Développement à base de composants et services web ż, construit la chaîne qui permet de passer de ces données brutes à un service web fiable et supervisé.

La chaîne comporte cinq parties, confiées chacune à un membre du groupe :

1. les **bases de données** : une base source MySQL contenant des données sales, une base cible PostgreSQL contenant des données propres, et un jeu de test de 40 lignes ;
2. l'**ETL** (extraction, transformation, chargement) qui alimente PostgreSQL à partir de MySQL ;
3. l'**API REST** Spring Boot qui expose les produits ;
4. la **supervision** de l'API avec Prometheus ;
5. les **tests** et la validation de bout en bout.

1.2 Principes directeurs

Trois principes ont guidé toutes les décisions techniques.

— **Reproductibilité** : tout se lance avec Docker Compose ; aucune installation manuelle d'une base de données ou d'un ETL n'est nécessaire.

— **Aucun secret dans le dépôt** : les mots de passe sont lus dans un fichier `.env` non versionné ; seul un modèle `.env.example` est partagé.

— **Contrôle de la qualité à chaque étape** : l'ETL rejette les données invalides, la base cible les refuse par des contraintes, et l'API les refuse par une validation explicite.

2 Architecture générale

2.1 Vue d'ensemble

La figure 1 montre les composants et les flux de données. Les données circulent de la gauche vers la droite : MySQL (source), ETL, PostgreSQL (cible), puis API. Prometheus interroge l'API à intervalle régulier. Tous les composants tournent dans des conteneurs Docker décrits par un unique fichier `docker-compose.yml`.

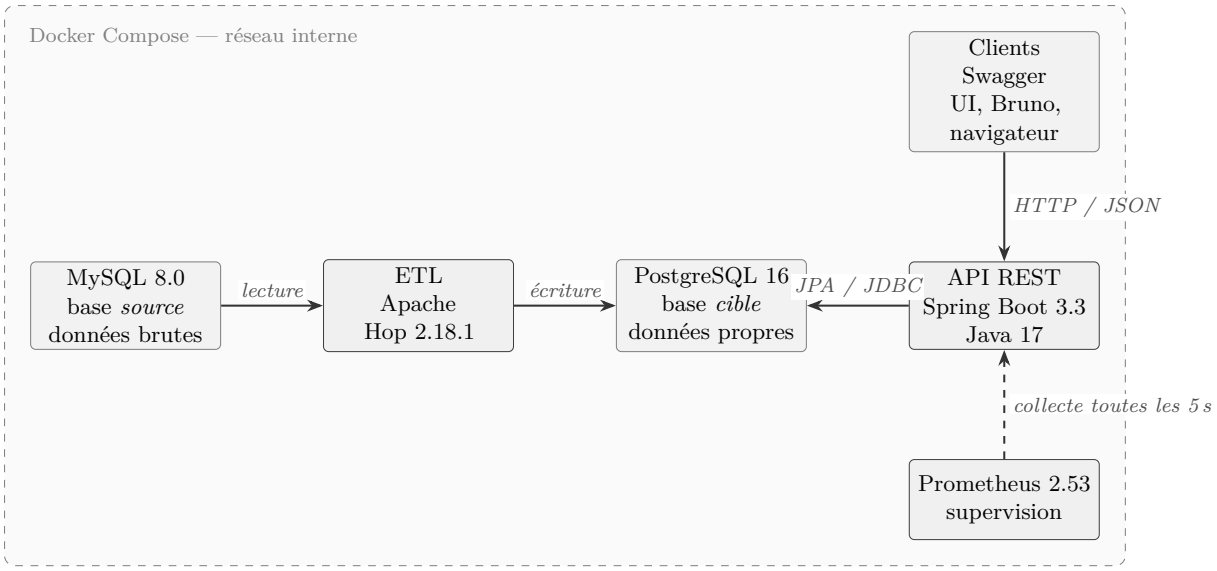


Figure 1 – Architecture générale du projet.

2.2 Les services Docker

Service	Image	Port hôte	Rôle
`mysql`	`mysql:8.0`	3306	Base source, initialisée par des scripts SQL
`postgres`	`postgres:16`	5432 (`PG_PORT`)	Base cible, utilisée par l'ETL et l'API
`etl`	Apache Hop 2.18.1 (image construite dans `etl/`)	aucun	Exécute le workflow puis s'arrête (à la demande)
`hop-web`	idem	8090	Interface graphique de Hop (optionnelle)
`api`	construite depuis `api/`	8080	API REST Spring Boot
`prometheus`	`prom/prometheus:v2.53.0`	9090	Collecte et consultation des métriques

Table 1 – Services définis dans `docker-compose.yml`.

Plusieurs mécanismes rendent le démarrage fiable :

— des **contrôles de santé** (*healthchecks*) sur MySQL et PostgreSQL ; l'API et l'ETL n'attendent que les bases soient `healthy` avant de démarrer (`depends_on` avec condition) ;

— des **volumes nommés** (`mysql_data`, `pg_data`) qui conservent les données entre deux arrêts ; `docker compose down -v` réinitialise tout ;

— des **scripts d'initialisation** montés dans `/docker-entrypoint-initdb.d`, exécutés automatiquement au premier démarrage, dans l'ordre alphabétique (`01-`, `02-`) ;

— des **ports configurables** dans `.env`, utiles lorsqu'une machine héberge déjà une base de données (voir la section 8).

2.3 Choix structurants

Le schéma est écrit en SQL, pas généré par Hibernate. L'API démarre avec `ddl-auto: validate` : si la table ne correspond pas au code, elle refuse de démarrer. Le même script SQL sert donc de référence à l'ETL, à l'API et aux tests.

Deux bases distinctes. La séparation source/cible reproduit un cas réel : la source est en lecture seule pour l'ETL et ne contient aucune contrainte, la cible impose toutes les règles.

Un seul réseau Docker. Les conteneurs se joignent par leur nom (`mysql`, `postgres`, `api`) ; seuls les ports utiles sont publiés sur la machine.

3 Partie 1 — Bases de données et jeu de test

3.1 Base source MySQL

La table `produits_source` est créée *sans aucune contrainte* (ni `NOT NULL`, ni `CHECK`), de façon volontaire : elle représente une source externe peu fiable. C'est à l'ETL de faire le contrôle qualité.

Colonne	Type	Remarque
`id`	`INT AUTO_INCREMENT`	Clé primaire
`nom`	`VARCHAR(150)`	Peut être vide ou nul
`description`	`TEXT`	
`prix`	`DECIMAL(10,2)`	Peut être négatif ou nul
`quantite`	`INT`	Peut être négative
`categorie`	`VARCHAR(100)`	Écrite de plusieurs façons (`electro`, `ÉLECTRONIQUE`...)
`date_creation`	`DATETIME`	`DEFAULT CURRENT_TIMESTAMP`

Table 2 – Structure de la base source.

La base est configurée en `utf8mb4` afin de stocker correctement les accents (ń Électronique ż, ń Vêtements ż).

3.2 Base cible PostgreSQL

La base cible contient deux tables et applique toutes les règles de gestion.

Table	Colonne	Contrainte
`produits`	`id`	`SERIAL PRIMARY KEY`
	`nom`	`VARCHAR(150) NOT NULL`
	`description`	`TEXT`
	`prix`	`NUMERIC(10,2) NOT NULL CHECK (prix >= 0)`
	`quantite`	`INTEGER NOT NULL DEFAULT 0 CHECK (quantite >= 0)`
	`categorie`	`VARCHAR(100) NOT NULL`
	`date_creation`	`TIMESTAMP NOT NULL DEFAULT NOW()`
`produits_rejets`	`id_source`	`INT` (identifiant dans MySQL)
	`motif`	`VARCHAR(255)` (raison du rejet)
	`date_rejet`	`TIMESTAMP DEFAULT NOW()`

Table 3 – Structure de la base cible.

Deux index accélèrent les requêtes fréquentes : un index sur `categorie` (filtre de l'API) et un index sur `(nom, categorie)`, qui correspond à la clé fonctionnelle utilisée par l'ETL pour détecter les doublons.

3.3 Jeu de test

Le jeu de test insère 40 produits, construits pour exercer chaque règle de l'ETL (tableau 4). Le fichier `db/DONNEES-TEST.md` détaille chaque ligne et sert de référence aux parties ETL et tests.

Lignes	Contenu	Résultat attendu
1 à 29	Produits valides, catégories écrites de plusieurs façons	Chargées dans `produits`
30	Doublon de la ligne 7 (*Souris sans fil*, Informatique)	Mise à jour de la ligne existante
31 à 38, 40	Nom, prix, quantité ou catégorie manquant ou invalide	Rejetées avec un motif
39	Catégorie inconnue (`jouets`)	Rejetée (décision du groupe)

Table 4 – Répartition des 40 lignes de test.

Le jeu contient aussi des cas limites *valides* qu'il ne faut pas rejeter : une quantité égale à zéro (ligne 21) et un prix égal à zéro (ligne 27).

Le script `db/verifier-bases.sh` contrôle que les deux bases répondent et affiche le nombre de lignes (40 dans la source ; les tables cibles sont vides avant le premier passage de l'ETL).

4 Partie 2 — ETL avec Apache Hop

4.1 Choix de l'outil

L'énoncé proposait Talend Open Studio. Cet outil n'est plus maintenu depuis 2024 et son installation n'est plus fiable ; l'énoncé autorisant une autre solution ETL, le groupe a retenu **Apache Hop 2.18.1** pour trois raisons : c'est un projet libre et actif de la fondation Apache (issu de Pentaho Kettle) ; ses traitements sont enregistrés en XML et donc versionnables avec Git ; il fournit une image Docker officielle, ce qui rend l'exécution reproductible.

4.2 Le pipeline de traitement

Le pipeline `pl_produits` traite chaque ligne en cinq étapes (figure 2).

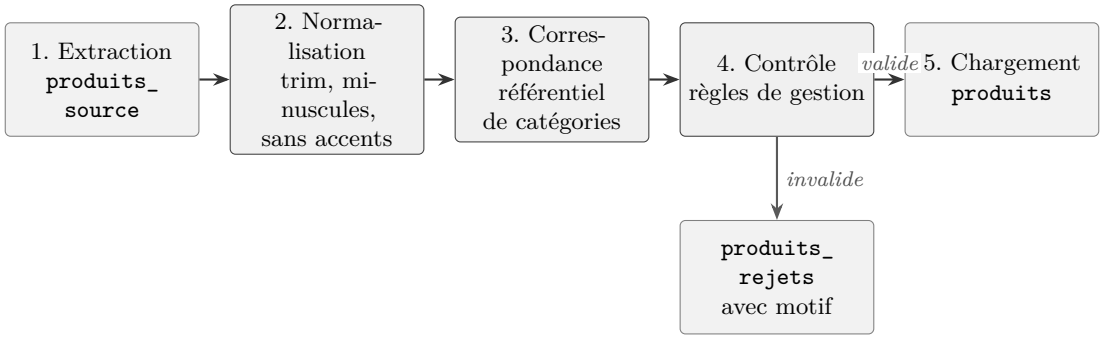


Figure 2 – Étapes du pipeline ETL.

1. **Extraction** : lecture de la table `produits_source` dans MySQL.
2. **Normalisation** : suppression des espaces superflus dans le nom ; mise en minuscules de la catégorie, suppression des espaces et des accents (ń ÉLECTRONIQUE ż devient ń electronique ż).

3. **Correspondance** : recherche de la catégorie officielle dans le référentiel `categories.csv` (ń electro ż devient ń Electronique ż). Il y a cinq catégories officielles : Electronique, Informatique, Vetements, Alimentation et Maison.
4. **Contrôle** : le nom, le prix et la catégorie sont obligatoires ; le prix et la quantité doivent être positifs ou nuls ; la catégorie doit être connue. Chaque ligne invalide reçoit un motif précis (par exemple ń prix négatif ż ou ń catégorie inconnue (jouets) ż).
5. **Chargement** : les lignes valides sont insérées dans `produits`, ou mises à jour si un produit de même nom et de même catégorie existe déjà, ce qui évite les doublons. Les lignes invalides sont écrites dans `produits_rejets`.

4.3 Le workflow d'orchestration

Le workflow `wf_etl_produits` enchaîne la vérification des connexions, l'exécution du pipeline, le calcul d'un bilan (pipeline `pl_bilan`) puis la journalisation. En cas d'erreur, il écrit un message d'échec et se termine avec un code de sortie non nul : l'ETL peut ainsi être intégré à un test automatique.

4.4 Exécution et résultats

```
docker compose up -d            # démarre les bases
docker compose run --rm etl     # exécute le workflow puis s'arrête
```

Indicateur	Valeur
Lignes lues dans MySQL	40
Lignes valides	30
Produits chargés dans PostgreSQL	29
Lignes rejetées	10

Table 5 – Résultats de l'ETL sur le jeu de test (source : rapport de la partie ETL, `etl/RAPPORT-ETL.md`).

Les 10 lignes rejetées portent chacune un motif précis, relevé dans la table `produits_rejets` :

Ligne source	Motif de rejet
31, 32, 33	nom manquant (valeur nulle, vide, ou espaces uniquement)
34	prix manquant
35	prix négatif
36	quantité négative
37, 38	catégorie manquante (valeur nulle ou vide)
39	catégorie inconnue (jouets)
40	nom manquant, prix manquant

Table 6 – Motifs de rejet observés après exécution de l'ETL.

L'écart entre 30 lignes valides et 29 produits chargés s'explique par le doublon : la ligne 30 met à jour ń Souris sans fil ż au lieu de créer une nouvelle ligne. La catégorie ń jouets ż, absente du référentiel, est rejetée. Une seconde exécution donne les mêmes chiffres : l'ETL est *idempotent*, car la table des rejets est vidée à chaque passage et le chargement fonctionne par insertion ou mise à jour.

Les identifiants des bases ne sont écrits dans aucun fichier Hop : les connexions utilisent des variables (`${MYSQL_PASSWORD}`, `${PG_PASSWORD}`…) fournies par `.env` au démarrage du conteneur.

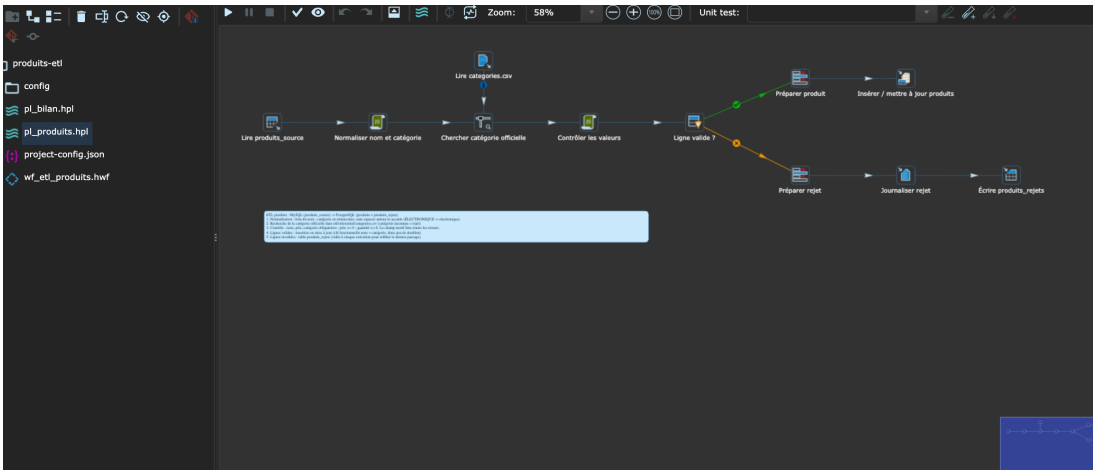


Figure 3 – Le pipeline `pl_produits` dans l'interface Hop Web.

5 Partie 3 — API REST Spring Boot

5.1 Technologies

Élément	Choix
Langage et framework	Java 17, Spring Boot 3.3.5
Accès aux données	Spring Data JPA, Hibernate, pilote PostgreSQL
Validation	Jakarta Bean Validation (`@Valid`, `@NotBlank`, `@PositiveOrZero`…)
Documentation	springdoc-openapi 2.6.0 (contrat OpenAPI et Swagger UI)
Supervision	Spring Boot Actuator et Micrometer (registre Prometheus)
Confort de code	Lombok (accesseurs, constructeurs)
Tests	JUnit 5, Mockito, MockMvc, base H2 en mode PostgreSQL

Table 7 – Pile technique de l'API.

5.2 Architecture en couches

Le code suit une architecture en couches classique (figure 4), chaque couche n'appelant que la suivante.

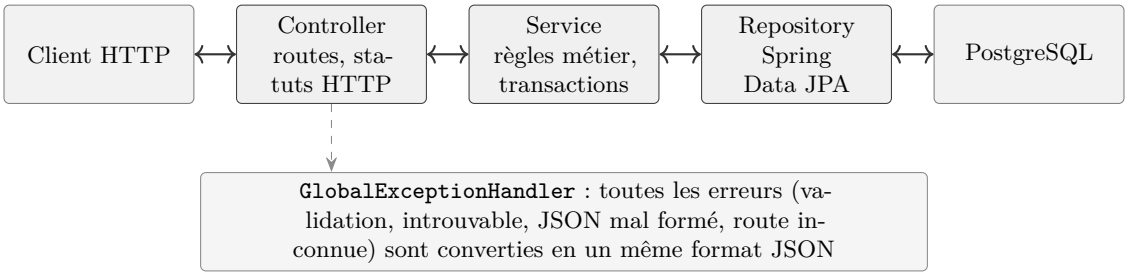


Figure 4 – Couches de l'API.

`entity` La classe `Produit` est liée à la table `produits`. Le champ `dateCreation` est posé par la base (`DEFAULT NOW()`) et relu après l'insertion ; l'API ne l'écrit jamais, ce qui garantit

qu'un `PUT` ne peut pas la modifier.

`dto` Des objets de transfert séparent le contrat de l'API du modèle de données : `ProduitRequest` (entrée, avec les règles de validation), `ProduitResponse` (sortie), `PageResponse` (résultat paginé) et `SuppressionResponse` (confirmation de suppression). Ce sont des *records* Java, donc immuables.

`repository` Interface Spring Data ; elle ajoute la recherche par catégorie sans tenir compte de la casse.

`service` Contient la logique et les transactions : lecture seule par défaut, écriture explicite pour la création, la modification et la suppression.

`controller` Déclare les routes sous `/api/v1/produits` et documente chaque réponse pour OpenAPI. La route `/` redirige vers Swagger UI.

`exception` Un gestionnaire centralisé (`@RestControllerAdvice`) produit toutes les réponses d'erreur.

5.3 Les endpoints

Opération	Méthode	URL	Réponses
Lister	GET	`/api/v1/produits`	200 ; 400 si `page` < 0 ou `size` hors de 1 à 100
Consulter	GET	`/api/v1/produits/{id}`	200 ; 404
Créer	POST	`/api/v1/produits`	201 avec l'en-tête `Location` ; 400
Modifier	PUT	`/api/v1/produits/{id}`	200 ; 400 ; 404
Supprimer	DELETE	`/api/v1/produits/{id}`	200 avec message de confirmation ; 404

Table 8 – Ressource `produits`.

La liste accepte les paramètres `categorie` (filtre, casse ignorée), `page` (défaut 0) et `size` (défaut 20, maximum 100). Les résultats sont triés par identifiant et renvoyés sous la forme `{content, page, size, totalElements, totalPages}`.

Trois endpoints techniques complètent l'API : `/actuator/health` (état de l'application), `/actuator/prometheus` (métriques) et `/v3/api-docs` (contrat OpenAPI).

5.4 Validation et gestion des erreurs

Les règles de validation sont portées par `ProduitRequest` : le nom est obligatoire et limité à 150 caractères ; le prix est obligatoire, positif ou nul, avec au plus deux décimales ; la quantité est obligatoire, positive ou nulle ; la catégorie est obligatoire et limitée à 100 caractères. Les paramètres de pagination sont validés de la même façon.

Toutes les erreurs ont le **même format JSON**, y compris pour une route inconnue ou un corps JSON mal formé, ce qui simplifie le travail des applications clientes :

```
{"timestamp":"2026-10-28T15:36:52Z","status":400,"error":"Bad Request",
 "message":"Données invalides","path":"/api/v1/produits",
 "details":[{"champ":"prix","message":"Le prix doit être positif ou nul"}]}
```

Cause	Statut	Message
Corps invalide (champ vide, prix négatif…)	400	ń Données invalides ż + liste `details`
Paramètre de pagination hors limites	400	ń Paramètres invalides ż + `details`
Identifiant non numérique (`/produits/abc`)	400	ń Valeur invalide pour le paramètre 'id' ż
Corps JSON absent ou mal formé	400	ń Corps de requête JSON absent ou mal formé ż
Produit inexistant	404	ń Produit introuvable avec l'id …ż
Route inconnue	404	ń Ressource introuvable : …ż

Table 9 – Erreurs gérées par `GlobalExceptionHandler`.

5.5 Contrat OpenAPI

Le contrat est généré automatiquement par springdoc à partir du code puis *exporté dans le dépôt* (`api/openapi.yaml`) par un test dédié. Le contrat versionné permet de voir en une revue de code ce qu'un changement modifie pour les clients. La documentation interactive est disponible sur `/swagger-ui/index.html`.

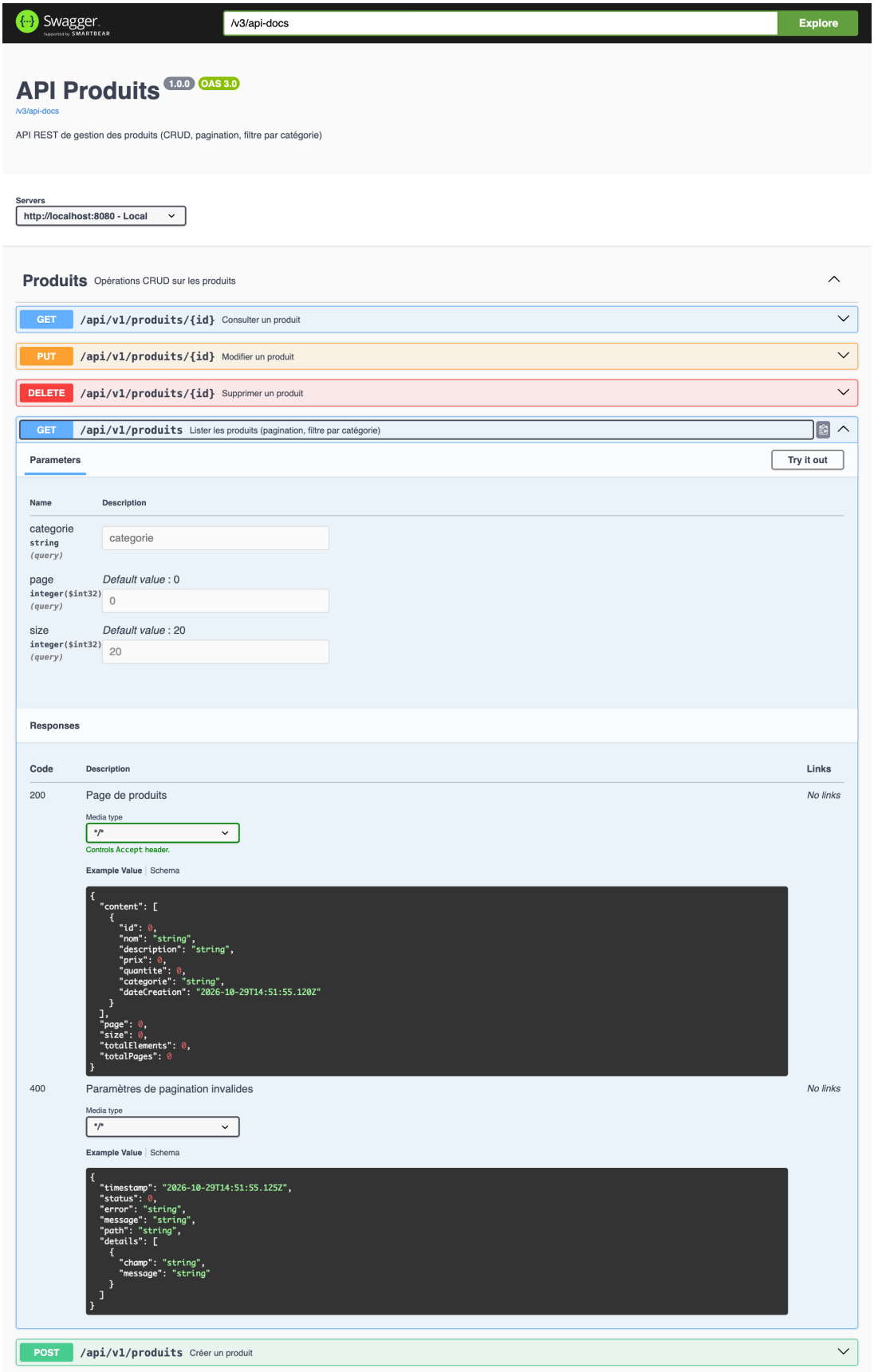


Figure 5 – Documentation interactive de l'API (Swagger UI).

5.6 Deux améliorations apportées aux réponses

Après une première version, deux réponses jugées trop pauvres ont été enrichies.

— **La suppression** renvoyait un `204` sans corps. Elle renvoie désormais un `200` avec un message de confirmation et le produit tel qu'il était avant suppression :

```
{"message":"Produit 11 supprimé",
 "produit":{"id":11,"nom":"Souris","prix":19.90,"quantite":40,
            "categorie":"Informatique","dateCreation":"2026-10-28T15:36:52"}}
```

Ce changement modifie le contrat (le statut passe de 204 à 200) ; les tests, le contrat OpenAPI, le README et la collection Bruno ont été mis à jour en conséquence.

— **L'état de santé** (`/actuator/health`) ne renvoyait que `{"status":"UP"}`. Avec `show-details: always`, il détaille chaque composant : base PostgreSQL, espace disque libre et *ping*. Un incident de base de données devient visible sans consulter les journaux.

5.7 Conteneurisation

Le `Dockerfile` de l'API a deux étapes : une image Maven compile l'application, puis seule l'archive `.jar` est copiée dans une image *JRE 17* légère, exécutée par un utilisateur non privilégié. Les dépendances sont téléchargées dans une couche séparée, ce qui accélère les reconstructions.

6 Partie 4 — Supervision avec Prometheus

6.1 Principe

L'API expose ses métriques au format Prometheus sur `/actuator/prometheus`, grâce à Actuator et Micrometer. Le serveur Prometheus les *collecte* (*scrape*) toutes les 5 secondes, les stocke et permet de les interroger avec le langage PromQL, sur `http://localhost:9090`.

```
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["prometheus:9090"]
  - job_name: "produits-api"
    metrics_path: "/actuator/prometheus"
    static_configs:
      - targets: ["api:8080"]
```

La cible de l'API est désignée par son nom de service Docker (`api:8080`) : la configuration est donc identique sur toutes les machines. Dans l'interface de Prometheus (page *Status* > *Targets*), les deux cibles apparaissent à l'état `UP`.

6.2 Métriques disponibles

Métrique	Information
`http_server_requests_seconds_*`	Nombre et durée des requêtes, par méthode, URI, statut et résultat
`jvm_memory_used_bytes`	Mémoire utilisée (tas et hors-tas)
`hikaricp_connections_*`	Connexions actives et en attente vers PostgreSQL
`process_cpu_usage`	Utilisation du processeur
`up`	1 si la cible répond à la collecte, 0 sinon

Table 10 – Principales familles de métriques.

6.3 Requêtes PromQL utiles

```
up                                                    # état de chaque cible
sum by (uri) (rate(http_server_requests_seconds_count{uri=~"/api/.*"}[5m]))
                                                      # requêtes par seconde et par route
sum by (status) (http_server_requests_seconds_count{uri=~"/api/.*"})
                                                      # nombre de réponses par code HTTP
sum by (area) (jvm_memory_used_bytes)                 # mémoire de la JVM
hikaricp_connections_active                           # connexions actives à la base
```

Ces requêtes ont été exécutées sur l'environnement du projet et renvoient des séries de données. Elles permettent par exemple de repérer une hausse des erreurs 4xx/5xx ou une saturation de la mémoire.

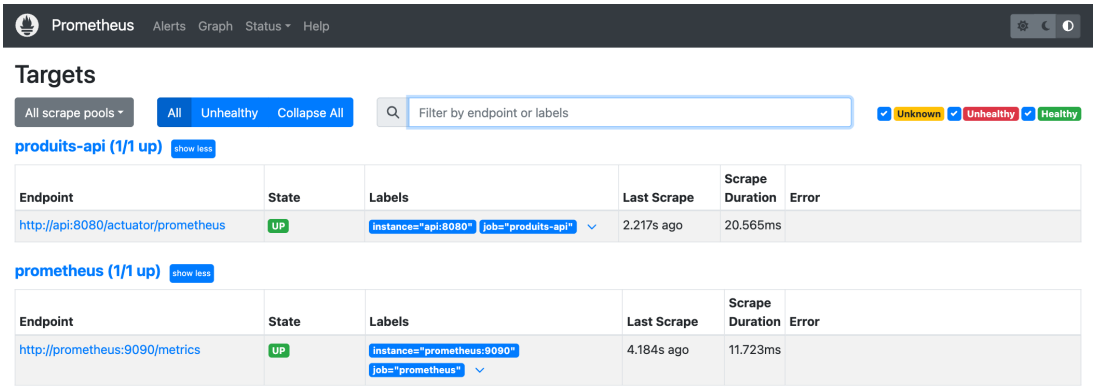


Figure 6 – Cibles de collecte dans Prometheus.

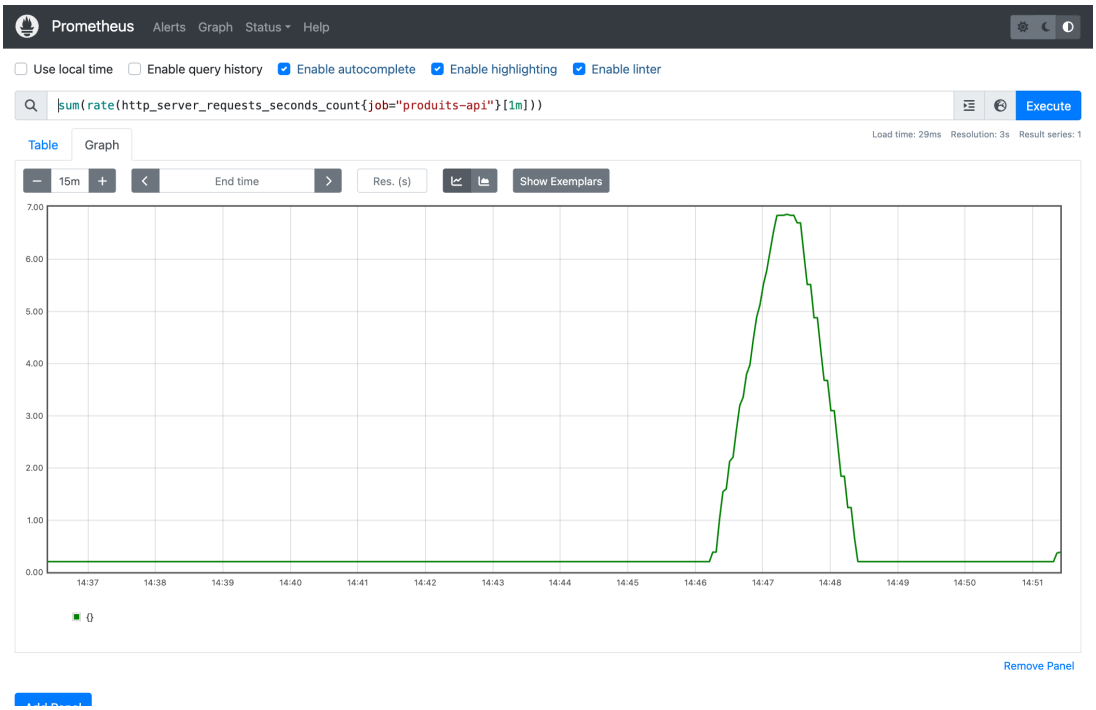


Figure 7 – Requête PromQL sur le trafic de l'API.

7 Partie 5 — Tests et validation

La qualité est vérifiée à quatre niveaux, du plus fin au plus global : tests Java (unitaires et d'intégration), collections Postman et Bruno sur l'API en marche, puis script de bout en bout sur toute la chaîne.

7.1 Tests unitaires du service

Neuf tests JUnit 5 isolent la couche service en simulant le repository avec Mockito. Ils vérifient la création (avec normalisation des espaces), la modification, la suppression (y compris la valeur renvoyée), la liste avec ou sans filtre de catégorie, et le déclenchement de `ProduitNotFoundException` quand un produit est absent.

7.2 Tests d'intégration de l'API

Dix-huit tests parcourent l'application de bout en bout — contrôleur, validation, service, JPA — avec MockMvc et une base H2 configurée en mode PostgreSQL. Le schéma de test (`src/test/resources/schema-test.sql`) reproduit celui de PostgreSQL, y compris les contraintes `CHECK`. Les cas couverts sont notamment :

- création valide (201, en-tête `Location`, date de création posée) ;
- création invalide (400, champs signalés), prix manquant et JSON mal formé ;
- modification, dont la conservation de la date de création, et modification invalide ;
- suppression (200 avec confirmation), puis disparition du produit (404) ;
- consultation d'un produit absent (404) et identifiant non numérique (400) ;
- pagination, filtre de catégorie, paramètres de pagination hors limites ;
- route inconnue (404 au format JSON), redirection de la racine vers Swagger, exposition du contrat OpenAPI et de l'état de santé.

Ces 27 tests (9 + 18) s'exécutent sans Docker avec `mvn test` ; ils réussissent tous. Un 28e test, qui exporte le contrat OpenAPI, est désactivé par défaut et se lance avec `-Dopenapi.export=true`. Le projet cible Java 17 : avec une version plus récente (par exemple Java 27), la compilation échoue à cause de Lombok.

7.3 Collection Bruno

La collection du dossier `bruno/` teste l'API réelle, en marche, par HTTP. Elle contient **18 requêtes** en trois dossiers, chacune avec des assertions et des tests aux messages explicites.

Dossier	Requêtes	Contenu
Produits	7	Lister, lister par catégorie, créer, consulter, modifier, supprimer, vérifier la suppression
Erreurs	8	Données invalides (création et modification), pagination invalide, identifiant non numérique, produit inexistant (consulter, modifier, supprimer), route inconnue
Supervision	3	Santé détaillée, métriques Prometheus, contrat OpenAPI

Table 11 – Organisation de la collection Bruno.

Le dossier *Produits* forme un scénario : la requête de création mémorise l'identifiant du produit dans une variable, que les requêtes suivantes réutilisent. Le parcours complet crée puis supprime un produit, et laisse donc la base dans son état initial. La collection s'ouvre dans Bruno par *Open Collection*, avec l'environnement *Local* (`baseUrl = http://localhost:8080`).

7.4 Collection Postman

L'énoncé demande de tester l'API avec Postman. Le dossier `postman/` contient donc la même collection sous forme de fichier Postman (`API-Produits.postman_collection.json`) : mêmes 18 requêtes, mêmes trois dossiers, mêmes vérifications. Elle s'importe dans Postman (*Import*) et se lance avec le *Collection Runner*, ou en ligne de commande avec `newman`. La variable `baseUrl` vaut `http://localhost:8080` et la requête de création mémorise l'identifiant du produit, comme dans Bruno. Exécutée avec `newman` sur l'API en marche le 29 octobre 2026 : **18 requêtes, 41 assertions, 0 échec**.

7.5 Test de bout en bout

Le test de bout en bout vérifie que toute la chaîne fonctionne ensemble, depuis une machine vierge. Le scénario est le suivant ; les valeurs attendues proviennent du jeu de test (section 3.3).

Nř	Action	Résultat attendu
1	`docker compose down -v` puis `docker compose up -d -build`	4 services démarrés ; MySQL et PostgreSQL `healthy`
2	`bash db/verifier-bases.sh`	40 lignes dans MySQL ; `produits` et `produits_rejets` vides
3	`docker compose run -rm etl`	Code de sortie 0 ; 40 lues, 29 chargées, 10 rejetées
4	`GET /api/v1/produits?size=100`	`totalElements = 29`
5	`GET /api/v1/produits?categorie=informatique`	Uniquement des produits ń Informatique ż, quelle que soit la casse demandée
6	Dossier *Produits* de Bruno	Création, lecture, modification, suppression, puis 404
7	Dossier *Erreurs* de Bruno	Tous les cas d'erreur renvoient le format JSON standard
8	Page *Targets* de Prometheus	Cible `produits-api` à l'état `UP` ; compteurs des requêtes en hausse
9	Second passage de l'ETL	Mêmes chiffres (29 produits), sans doublon

Table 12 – Scénario de bout en bout et résultats attendus.

Résultats observés. Le scénario a été exécuté le 28 octobre 2026 sur une machine neuve côté Docker (volumes supprimés par `docker compose down -v`). Les 9 étapes réussissent (tableau 13).

Nř	Résultat observé	Statut
1	Les 4 services permanents démarrent en une vingtaine de secondes ; MySQL et PostgreSQL sont `healthy` après environ 9 s ; l'API répond `UP`.	**OK**
2	MySQL contient 40 lignes ; `produits` et `produits_rejets` sont vides ; l'API renvoie `totalElements = 0`.	**OK**
3	Code de sortie 0. Journal : 40 lignes lues, 29 produits chargés, 10 rejetées. Le workflow dure 1,6 s ; la commande complète 1 min 23 s à la première exécution (construction de l'image Hop comprise).	**OK**
4	`totalElements = 29`, produits triés par identifiant croissant.	**OK**
5	Le filtre renvoie 6 produits ń Informatique ż pour `informatique`, `INFORMATIQUE` et `Informatique`. Répartition des 29 produits : Alimentation 5, Electronique 7, Informatique 6, Maison 5, Vetements 6 ; aucun produit ń jouets ż.	**OK**
6	Parcours création, lecture, modification, suppression : 201 avec `Location`, 200, 200 (`dateCreation` inchangée), 200 avec message ń Produit 31 supprimé ż, puis 404 ; la base retrouve ses 29 produits. 9 vérifications.	**OK**
7	Les 8 cas d'erreur renvoient le statut attendu (400 ou 404) et le format JSON standard. 8 vérifications.	**OK**
8	Cibles `produits-api` et `prometheus` à l'état `UP` ; le compteur des requêtes sur `/api` passe de 29 à 31 après deux requêtes supplémentaires.	**OK**
9	Code de sortie 0 ; mêmes chiffres (40, 29, 10) ; l'API liste toujours 29 produits, soit 29 couples (nom, catégorie) distincts : aucun doublon. Commande complète : 18 s.	**OK**

Table 13 – Résultats du scénario de bout en bout (28 vérifications automatisées sur les étapes 4 à 7 et la supervision, 0 échec).

Les étapes 4 à 7 ont été rejouées avec la collection Bruno exécutée en ligne de commande (`bru run -env Local`, dossiers *Produits*, *Erreurs* et *Supervision*) : **18 requêtes réussies, 33**

tests et 19 assertions sans échec.

7.6 Script automatisé de bout en bout

Pour que ce scénario soit rejouable par n'importe quel relecteur, il a été automatisé dans le script `tests/e2e.sh`. Il suffit de lancer `bash tests/e2e.sh -clean` : le script réinitialise les volumes Docker, démarre les services, exécute l'ETL, interroge l'API et Prometheus, puis affiche `[OK]` ou `[KO]` pour chacun des 32 contrôles. Son code de retour vaut 0 si tout réussit, ce qui le rend utilisable dans une chaîne d'intégration continue. Contrairement aux collections Postman et Bruno, qui ne testent que l'API, il vérifie aussi directement le contenu des deux bases de données.

Étape	**Contrôles**	**Résultat**
Démarrage et source	API `UP` ; 40 lignes dans `produits_source`	2 OK
ETL	Code de sortie 0 ; 29 produits chargés ; 10 rejets, tous avec un motif ; aucune catégorie hors référentiel ; aucun prix négatif ; relance sans doublon (toujours 29)	7 OK
Lecture par l'API	Liste (`totalElements = 29`) ; filtre par catégorie égal au résultat de la requête SQL	4 OK
Création, modification, suppression	201 ; produit présent dans PostgreSQL ; 200 ; nom modifié en base ; 200 ; 404 après suppression ; produit absent de PostgreSQL ; retour à 29 produits	10 OK
Erreurs	Corps invalide (400, message standard) ; identifiant inexistant (404) ; identifiant non numérique (400) ; pagination invalide (400)	5 OK
Supervision	`/actuator/prometheus` (200, métrique HTTP présente) ; cible `produits-api` à `up = 1` ; requêtes HTTP comptées par Prometheus	4 OK

Table 14 – Contrôles du script `tests/e2e.sh` (le détail est dans `tests/RESULTATS-ATTENDUS.md`).

Résultat observé. Exécuté le 29 octobre 2026 avec l'option `-clean` (volumes supprimés, images reconstruites) : **32 contrôles réussis, 0 échec**.

8 Difficultés rencontrées et solutions

Difficulté	Solution
Talend Open Studio n'est plus maintenu (2024)	Choix d'Apache Hop, documenté et justifié, conforme à l'énoncé qui autorise une alternative
Apache Hop ne fournit pas le pilote MySQL (licence)	Pilote ajouté automatiquement à la construction de l'image Docker
Aucun composant Hop ne supprime les accents	Court script JavaScript intégré au pipeline
L'image officielle de Hop affichait les mots de passe dans son journal	Identifiants transmis par un fichier temporaire créé au démarrage du conteneur
Ports déjà occupés sur certaines machines (PostgreSQL local sur 5432, autre conteneur sur 5433)	Ports publiés configurables dans `.env` (`PG_PORT`) ; l'API, dans Docker, joint toujours `postgres:5432` en interne
Compilation Maven en échec avec Java 27 (Lombok incompatible)	Utilisation du JDK 17 visé par le projet ; l'image Docker n'est pas concernée
Réponses de l'API jugées trop pauvres (suppression sans corps, santé réduite à `UP`)	Suppression avec confirmation et santé détaillée, tests et contrat mis à jour

Table 15 – Principales difficultés et réponses apportées.

9 Organisation du travail

Le travail a été réparti en cinq parties, une par membre. Le dépôt Git applique un flux simple : une branche par fonctionnalité (`feat/*`), intégrée dans `main` par une demande de fusion (*pull request*).

Partie	Membre	Réalisations	Branche Git
1	BOUGOUMA Leïla	Scripts SQL des deux bases, jeu de test de 40 lignes (`DONNEES-TEST.md`), `docker-compose.yml`, `.env.example`, script `verifier-bases.sh`	`main` (commit initial)
2	OUARMA Landry	Projet Apache Hop (pipelines `pl_produits` et `pl_bilan`, workflow `wf_etl_produits`), référentiel de catégories, services Docker `etl` et `hop-web`, installation et rapport de l'ETL	`feat/etl-hop` (PR #3)
3	WAONGO Osiris	API Spring Boot en couches, validation, gestion centralisée des erreurs, contrat OpenAPI, `Dockerfile`, tests unitaires et d'intégration	`feat/api-dev` (PR #2)
4	ZAPRE Malika	Fichier `prometheus.yml`, service Prometheus dans Docker Compose, exposition des métriques de l'API	`feat/monitoring-prometheus` (PR #1)
5	KOURA Stéphane	Collection Bruno (18 requêtes), enrichissement des réponses de l'API (suppression, santé détaillée), scénario de bout en bout, rapport LaTeX et guide de présentation	`feat/tests` (PR #4)

Table 16 – Répartition des parties.

10 Limites et perspectives

— **Sécurité** : l'API n'a pas d'authentification, et l'état de santé détaillé est public. C'est acceptable pour un environnement de développement ; en production, il faudrait ajouter Spring Security (jetons JWT) et limiter l'accès aux endpoints `/actuator`.

— **ETL automatique** : l'ETL se lance à la demande. Il pourrait être planifié (tâche périodique) ou déclenché à l'arrivée de nouvelles données.

— **Supervision** : ajouter des règles d'alerte (taux d'erreurs, indisponibilité) avec Alertmanager, et des tableaux de bord Grafana.

— **Tests** : utiliser Testcontainers pour tester l'API contre un vrai PostgreSQL plutôt que H2, et exécuter le script `tests/e2e.sh` dans une chaîne d'intégration continue à chaque modification.

— **Évolution de l'API** : versionnement déjà prévu par le préfixe `/api/v1` ; ajout possible de la recherche par nom, du tri configurable et de la suppression logique.

11 Conclusion

Le projet atteint son objectif : une chaîne complète, reproductible et vérifiée, qui transforme des données brutes et imparfaites en un service web fiable et supervisé. L'ETL garantit la qualité des données, la base cible la protège par des contraintes, l'API l'expose avec un contrat clair et des erreurs homogènes, et Prometheus permet de suivre son comportement. La combinaison de Docker Compose, d'un contrat OpenAPI versionné, de 27 tests automatisés, de collections Postman et Bruno et d'un script de bout en bout rend le projet facile à lancer, à relire et à

faire évoluer. Le travail en groupe, avec des parties indépendantes reliées par des conventions communes (schéma SQL, variables d'environnement, ports), a permis d'avancer en parallèle sans blocage.

A Lancer le projet

```
# 1. Configuration (changer PG_PORT si le port 5432 est déjà pris)
cp .env.example .env

# 2. Démarrer MySQL, PostgreSQL, l'API et Prometheus
docker compose up -d --build
docker compose ps                       # attendre "healthy"

# 3. Charger les données propres dans PostgreSQL
docker compose run --rm etl

# 4. Vérifier
curl http://localhost:8080/actuator/health
curl "http://localhost:8080/api/v1/produits?size=100"

# 5. Test de bout en bout (32 contrôles attendus)
bash tests/e2e.sh
```

Adresse	Contenu
http://localhost:8080/swagger-ui/index.html	Documentation interactive de l'API
http://localhost:8080/actuator/health	État de l'application et de ses composants
http://localhost:9090	Prometheus
http://localhost:8090/ui	Hop Web (profil `hop-web`)

Table 17 – Adresses utiles.

B Structure du dépôt

```
.
|-- docker-compose.yml      services : mysql, postgres, etl, hop-web, api, prometheus
|-- .env.example            modèle de configuration (le .env n'est pas versionné)
|-- db/                     scripts SQL des deux bases, jeu de test, verifier-bases.sh
|-- etl/                    projet Apache Hop, référentiel de catégories, Dockerfile
|-- api/                    API Spring Boot : src/, pom.xml, Dockerfile, openapi.yaml
|-- monitoring/             prometheus.yml
|-- postman/                collection Postman (18 requêtes)
|-- bruno/                  collection Bruno (18 requêtes)
|-- tests/                  e2e.sh (bout en bout) et résultats attendus
|-- rapport/                ce rapport (LaTeX)
`-- docs/                   guide de préparation de la présentation
```

C Exemples de requêtes et de réponses

Créer un produit.

```
curl -X POST http://localhost:8080/api/v1/produits \
  -H 'Content-Type: application/json' \
```

```
  -d '{"nom":"Clavier","description":"AZERTY","prix":59.90,
       "quantite":25,"categorie":"Informatique"}'

HTTP/1.1 201
Location: http://localhost:8080/api/v1/produits/12
{"id":12,"nom":"Clavier","description":"AZERTY","prix":59.90,"quantite":25,
 "categorie":"Informatique","dateCreation":"2026-10-28T15:36:52.77"}
```

Lister avec filtre et pagination.

```
GET /api/v1/produits?categorie=informatique&page=0&size=2

{"content":[ {...}, {...} ],"page":0,"size":2,"totalElements":7,"totalPages":4}
```

Produit inexistant.

```
GET /api/v1/produits/999999            ->  404

{"timestamp":"2026-10-28T15:36:52Z","status":404,"error":"Not Found",
 "message":"Produit introuvable avec l'id 999999",
 "path":"/api/v1/produits/999999"}
```